DATA612 Project 2: Collaborative Filtering Recommendation Models

Chhiring Lama

This project uses the MovieLens dataset to build and compare two collaborative filtering recommendation approaches: User-User Collaborative Filtering and Item-Item Collaborative Filtering.

1. Import Libraries and Load Data

In this section, we import the required Python libraries and load the MovieLens datasets used throughout the project.

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [3]:
```
ratings = pd.read_csv("ratings.csv")
movies = pd.read_csv("movies.csv")

print("Ratings shape:", ratings.shape)
print("Movies shape:", movies.shape)
```

```
Ratings shape: (100836, 4)
Movies shape: (9742, 3)
```

3. Dataset Inspection

In this section, we examine the structure and quality of the MovieLens datasets.

In [4]:
```
print("Ratings Dataset")
display(ratings.head())

print("\nMovies Dataset")
display(movies.head())
```

```
Ratings Dataset
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

Movies Dataset

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

4. Data Quality Assessment

In this section, we check for missing values and duplicate records.

In [5]:
```
print("Missing Values in Ratings")
print(ratings.isnull().sum())

print("\nMissing Values in Movies")
print(movies.isnull().sum())

print("\nDuplicate Rows in Ratings:", ratings.duplicated().sum())
print("Duplicate Rows in Movies:", movies.duplicated().sum())
```

```
Missing Values in Ratings
userId       0
movieId      0
rating       0
timestamp    0
dtype: int64

Missing Values in Movies
movieId    0
title      0
genres     0
dtype: int64

Duplicate Rows in Ratings: 0
Duplicate Rows in Movies: 0
```

5. Dataset Summary

In this section, we summarize the number of users, movies, and ratings in the dataset.

In [6]:
```
print("Number of Ratings:", len(ratings))
print("Number of Users:", ratings["userId"].nunique())
print("Number of Movies Rated:", ratings["movieId"].nunique())
print("Number of Movies in Catalog:", movies["movieId"].nunique())
```

```
Number of Ratings: 100836
Number of Users: 610
Number of Movies Rated: 9724
Number of Movies in Catalog: 9742
```

6. Train-Test Split

In this section, the ratings dataset is divided into training and testing sets. The training set is used to build recommendation models, while the testing set is used to evaluate their performance.

In [7]:
```
from sklearn.model_selection import train_test_split

train_data, test_data = train_test_split(
    ratings,
    test_size=0.20,
    random_state=42
)

print("Training set shape:", train_data.shape)
print("Testing set shape:", test_data.shape)
```

```
Training set shape: (80668, 4)
Testing set shape: (20168, 4)
```

7. Training and Testing Data Summary

In this section, we summarize the training and testing datasets after the split.

In [8]:
```
print("Training Ratings:", len(train_data))
print("Testing Ratings:", len(test_data))

print("\nUnique Users in Training:", train_data["userId"].nunique())
print("Unique Users in Testing:", test_data["userId"].nunique())

print("\nUnique Movies in Training:", train_data["movieId"].nunique())
print("Unique Movies in Testing:", test_data["movieId"].nunique())
```

```
Training Ratings: 80668
Testing Ratings: 20168

Unique Users in Training: 610
Unique Users in Testing: 610

Unique Movies in Training: 8983
Unique Movies in Testing: 5142
```

8. Create User-Item Matrix

In this section, the training data is transformed into a user-item matrix that will be used for collaborative filtering.

In [9]:
```
user_item_matrix = train_data.pivot_table(
    index="userId",
    columns="movieId",
    values="rating"
)

print("User-Item Matrix Shape:", user_item_matrix.shape)

user_item_matrix.tail()
```

```
User-Item Matrix Shape: (610, 8983)
```

Out[9]:

movieId	1	2	3	4	5	6	7	8	9	10	...	191005	193565
userId													
606	2.5	NaN	NaN	NaN	NaN	NaN	2.5	NaN	NaN	NaN	...	NaN	NaN
607	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN
608	2.5	2.0	2.0	NaN	NaN	NaN	NaN	NaN	NaN	4.0	...	NaN	NaN
609	3.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.0	...	NaN	NaN
610	5.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN

5 rows × 8983 columns

9. Handling Missing Ratings

The user-item matrix contains many missing values because most users rate only a small number of movies. These missing values do not mean that a user disliked a movie; they simply mean that no rating is available. Therefore, this project keeps missing ratings as NaN instead of replacing them with zero.

10. User-User Collaborative Filtering

User-User Collaborative Filtering recommends movies by finding users with similar rating patterns. Similarity is calculated based only on movies that two users have both rated.

In [10]:
```
user_similarity = user_item_matrix.T.corr(method="pearson")
```

11. Identify Similar Users

For each user, the recommender identifies the most similar users based on Pearson correlation. These neighboring users are used to estimate unknown ratings.

In [11]:
```
def get_top_k_users(user_id, similarity_matrix, k=10):
    similar_users = similarity_matrix[user_id].dropna()
    similar_users = similar_users.drop(user_id, errors="ignore")
    return similar_users.sort_values(ascending=False).head(k)
```

12. Minimum Overlap Constraint

Pearson correlation can produce misleadingly high similarity scores when two users have rated only a few movies in common. To improve reliability, similarities are retained only when users share at least 10 commonly rated movies.

In [12]:
```
min_common = 10

user_similarity_filtered = user_similarity.copy()

for user1 in user_item_matrix.index:
    for user2 in user_item_matrix.index:

        if user1 == user2:
            continue

        common = (
            user_item_matrix.loc[user1].notna() &
            user_item_matrix.loc[user2].notna()
        ).sum()

        if common < min_common:
            user_similarity_filtered.loc[user1, user2] = np.nan
```

In [13]:
```
top_users_for_1 = get_top_k_users(1, user_similarity_filtered, k=10)

print("Top 10 Similar Users for User 1 After Minimum Overlap Filtering")
top_users_for_1
```

```
Top 10 Similar Users for User 1 After Minimum Overlap Filtering
```

Out[13]:
```
userId
297    0.874147
344    0.822182
445    0.683754
488    0.633827
596    0.628539
303    0.615291
201    0.605494
133    0.601593
178    0.599473
72     0.587137
Name: 1, dtype: float64
```

In [14]:
```
print(
    "Users with at least one valid neighbor:",
    (user_similarity_filtered.notna().sum(axis=1) > 1).sum()
)

print(
    "Average valid neighbors per user:",
    user_similarity_filtered.notna().sum(axis=1).mean()
)
```

```
Users with at least one valid neighbor: 604
Average valid neighbors per user: 171.68032786885246
```

13. User-User Rating Prediction

The User-User model predicts a user's rating for a movie by using ratings from the most similar users who rated that movie.

In [15]:
```
def predict_user_user_rating(user_id, movie_id, user_item_matrix, similarity_matrix

    if movie_id not in user_item_matrix.columns:
        return np.nan

    if user_id not in user_item_matrix.index:
        return np.nan

    similar_users = get_top_k_users(user_id, similarity_matrix, k)

    movie_ratings = user_item_matrix[movie_id]

    neighbor_ratings = movie_ratings.loc[similar_users.index]

    valid_neighbors = neighbor_ratings.dropna()

    if valid_neighbors.empty:
        return np.nan

    valid_similarities = similar_users.loc[valid_neighbors.index]

    predicted_rating = np.dot(valid_similarities, valid_neighbors) / valid_similari
```

```
    return predicted_rating
```

In [16]:
```
predict_user_user_rating(1, 1, user_item_matrix, user_similarity_filtered, k=25)
```

Out[16]: 4.076142525053787

14. User-User Model Evaluation

The User-User Collaborative Filtering model is evaluated using the test dataset. To improve prediction coverage, a fallback method is used when the User-User model cannot generate a rating from similar users. In those cases, the movie's average rating from the training data is used. If the movie average is unavailable, the global average rating from the training data is used.

In [17]:
```
actuals = []
predictions = []

global_mean = train_data["rating"].mean()
movie_means = train_data.groupby("movieId")["rating"].mean()

for _, row in test_data.iterrows():

    user_id = int(row["userId"])
    movie_id = int(row["movieId"])
    actual_rating = row["rating"]

    predicted_rating = predict_user_user_rating(
        user_id,
        movie_id,
        user_item_matrix,
        user_similarity_filtered,
        k=25
    )

    if np.isnan(predicted_rating):
        predicted_rating = movie_means.get(movie_id, global_mean)

    actuals.append(actual_rating)
    predictions.append(predicted_rating)

print("Total Test Ratings:", len(test_data))
print("Predictions Made:", len(predictions))
print("Predictions Skipped:", len(test_data) - len(predictions))
print("User-User Coverage (%):", round(len(predictions) / len(test_data) * 100, 2))
```

```
Total Test Ratings: 20168
Predictions Made: 20168
Predictions Skipped: 0
User-User Coverage (%): 100.0
```

15. User-User RMSE Evaluation

Root Mean Squared Error (RMSE) is used to measure the difference between predicted ratings and actual ratings in the test dataset.

In [18]:
```
from sklearn.metrics import mean_squared_error
from math import sqrt

rmse_user_user = sqrt(mean_squared_error(actuals, predictions))

print("User-User RMSE:", round(rmse_user_user, 4))
```

```
User-User RMSE: 1.147
```

16. Item-Item Collaborative Filtering

Item-Item Collaborative Filtering recommends movies based on similarities between items. Similarity is calculated by comparing rating patterns across users.

In [19]:
```
item_similarity = user_item_matrix.corr(method="pearson")
```

17. Item-Item Rating Prediction

The Item-Item model predicts a user's rating for a movie by looking at movies the user has already rated and comparing them to the target movie.

In [20]:
```
def predict_item_item_rating(user_id, movie_id, user_item_matrix, item_similarity,

    if user_id not in user_item_matrix.index:
        return np.nan

    if movie_id not in item_similarity.columns:
        return np.nan

    user_ratings = user_item_matrix.loc[user_id].dropna()

    if user_ratings.empty:
        return np.nan

    similar_items = item_similarity[movie_id].loc[user_ratings.index].dropna()

    if similar_items.empty:
        return np.nan

    top_similar_items = similar_items.sort_values(ascending=False).head(k)

    ratings_for_similar_items = user_ratings.loc[top_similar_items.index]

    predicted_rating = np.dot(top_similar_items, ratings_for_similar_items) / top_s

    return predicted_rating
```

In [21]:
```
predict_item_item_rating(1, 1, user_item_matrix, item_similarity, k=25)
```

Out[21]:
```
4.448314599602669
```

18. Item-Item Model Evaluation

The Item-Item Collaborative Filtering model is evaluated using the test dataset. RMSE is calculated using only cases where the model is able to generate a prediction.

In [22]:
```
actuals_item = []
predictions_item = []

for _, row in test_data.iterrows():

    user_id = int(row["userId"])
    movie_id = int(row["movieId"])
    actual_rating = row["rating"]

    predicted_rating = predict_item_item_rating(
        user_id,
        movie_id,
        user_item_matrix,
        item_similarity,
        k=25
    )

    if not np.isnan(predicted_rating):
        actuals_item.append(actual_rating)
        predictions_item.append(predicted_rating)

print("Total Test Ratings:", len(test_data))
print("Predictions Made:", len(predictions_item))
print("Predictions Skipped:", len(test_data) - len(predictions_item))
```

```
Total Test Ratings: 20168
Predictions Made: 18545
Predictions Skipped: 1623
```

In [23]:
```
rmse_item_item = sqrt(
    mean_squared_error(actuals_item, predictions_item)
)

print("Item-Item RMSE:", round(rmse_item_item, 4))
```

```
Item-Item RMSE: 1.2638
```

In [24]:
```
results = pd.DataFrame({
    "Model": ["User-User CF", "Item-Item CF"],
    "RMSE": [rmse_user_user, rmse_item_item],
    "Coverage (%)": [
        len(predictions) / len(test_data) * 100,
        len(predictions_item) / len(test_data) * 100
    ]
})
```

```
results["RMSE"] = results["RMSE"].round(4)
results["Coverage (%)"] = results["Coverage (%)"].round(2)

results
```

Out[24]:

	Model	RMSE	Coverage (%)
0	User-User CF	1.1470	100.00
1	Item-Item CF	1.2638	91.95

In [25]:

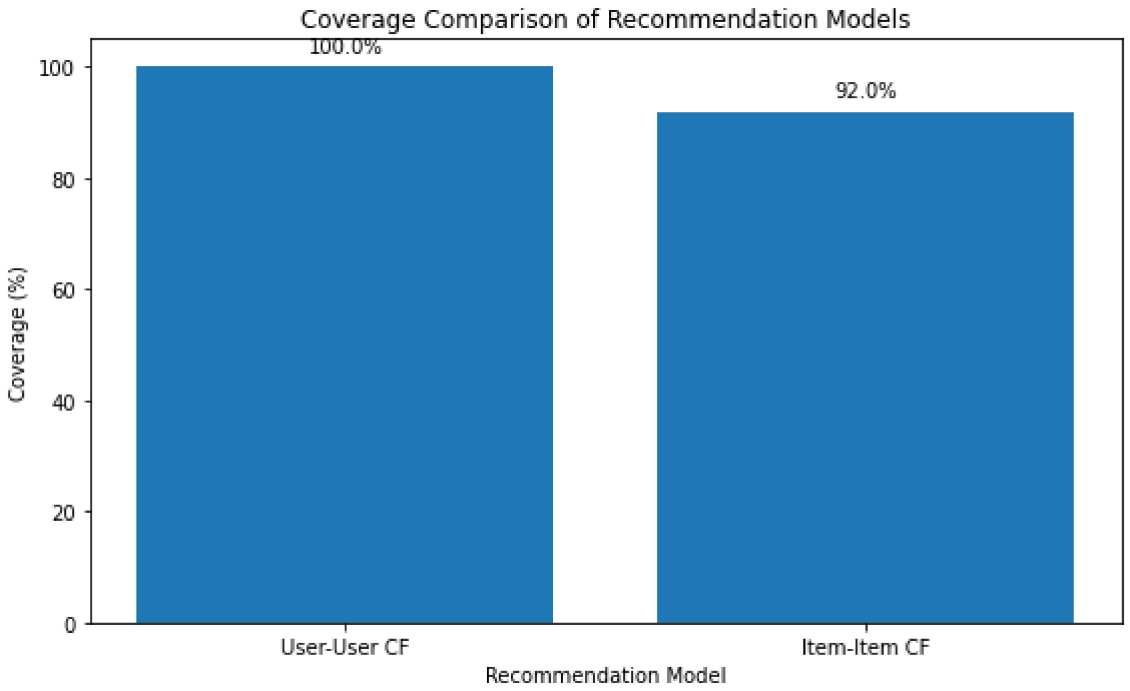
```
fig, ax = plt.subplots(figsize=(8, 5))

bars = ax.bar(results["Model"], results["Coverage (%)"])

ax.set_title("Coverage Comparison of Recommendation Models")
ax.set_xlabel("Recommendation Model")
ax.set_ylabel("Coverage (%)")
ax.set_ylim(0, 105)

for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height + 2,
        f"{height:.1f}%",
        ha="center",
        va="bottom"
    )

plt.tight_layout()
plt.show()
```



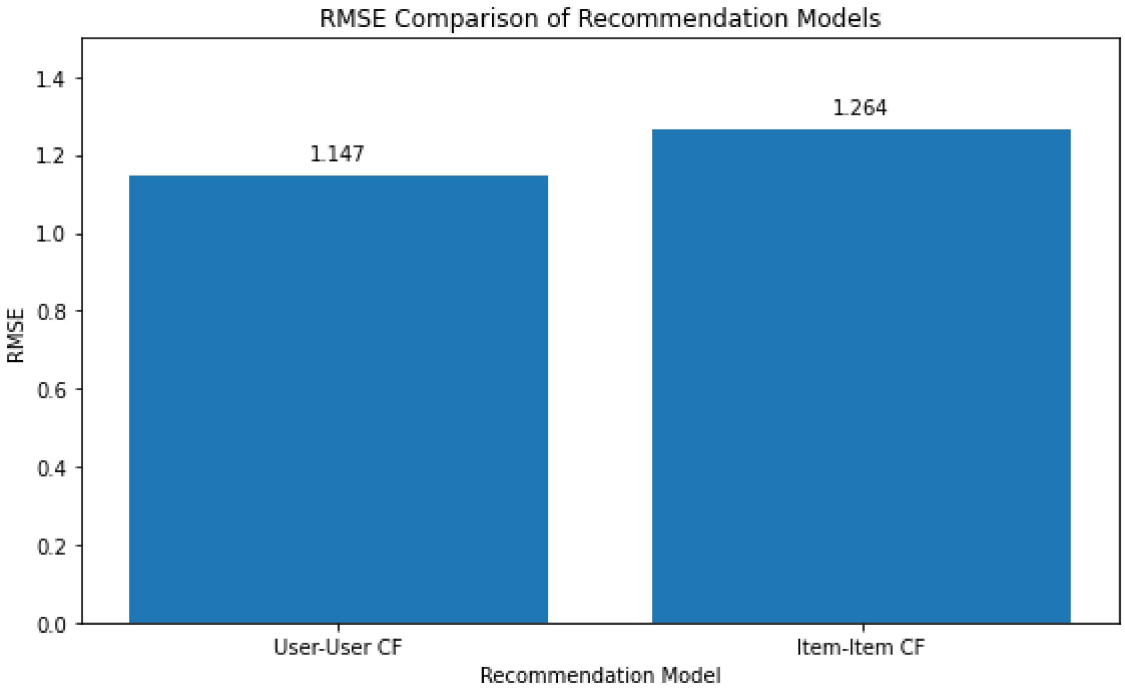
In [26]:
```
fig, ax = plt.subplots(figsize=(8, 5))

bars = ax.bar(results["Model"], results["RMSE"])

ax.set_title("RMSE Comparison of Recommendation Models")
ax.set_xlabel("Recommendation Model")
ax.set_ylabel("RMSE")
ax.set_ylim(0, 1.5)

for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.03,
        f"{height:.3f}",
        ha="center",
        va="bottom"
    )

plt.tight_layout()
plt.show()
```



21. Conclusion

This project compared two collaborative filtering recommendation methods using the MovieLens dataset: User-User Collaborative Filtering and Item-Item Collaborative Filtering.

The original User-User model had lower prediction coverage because it could not generate predictions for many test ratings when enough similar users were not available. To address this issue, a fallback method was added. When the User-User model could not make a

prediction, the model used the movie's average rating from the training data. If the movie average was not available, it used the global average rating from the training data.

After adding the fallback method, the User-User model generated predictions for 100.0% of the test ratings and achieved an RMSE of 1.1470. The Item-Item model generated predictions for 91.95% of the test ratings and achieved an RMSE of 1.2638.

Based on these results, the improved User-User Collaborative Filtering model performed better overall because it produced full prediction coverage while also achieving a lower RMSE than the Item-Item model.